

Lab 3: Kubernetes Container Orchestration Lab Report

Nabin Ranabhat (078BCT050)

Department of Electronics and Computer Engineering

Pulchowk Engineering Campus

IOE, Tribhuvan University

Contents

1	Introduction	2
2	Objectives	2
3	Tools and Technologies Used	2
4	Background: Key Kubernetes Concepts	3
5	Implementation Details	3
5.1	Minikube Installation and Cluster Initialisation	3
5.2	Building and Pushing Docker Images	4
5.3	Kubernetes Manifest Files	5
5.3.1	MySQL Secret	6
5.4	Building and Pushing Docker Images	6
5.5	Kubernetes Manifest Files	7
5.5.1	MySQL Secret	8
5.5.2	Database StatefulSet and Service	8
5.5.3	Backend Deployment and Service	10
5.5.4	Frontend Deployment and Service	12
5.6	Deploying All Resources	13
5.7	Verifying Pod and Service Status	14
6	Findings and Observations	15
7	Technical Challenges Encountered	15
8	Comparison: Docker Compose vs. Kubernetes	16
9	Conclusion	16

1 Introduction

Kubernetes (K8s) is an open-source container orchestration platform that automates the deployment, scaling, and management of containerised applications. Building upon the foundational containerisation skills acquired in Lab 2, this lab extends the workflow by deploying the same multi-service application—frontend (Nginx), backend (Flask/Python), and database (MySQL)—onto a local Kubernetes cluster managed by **Minikube**.

Unlike Docker Compose, which targets single-host environments, Kubernetes introduces abstractions such as *Pods*, *Deployments*, *Services*, and *ConfigMaps* that are designed for production-grade, multi-node deployments. This practical exercise covers the complete Kubernetes workflow from cluster initialisation to service exposure.

2 Objectives

- Install and configure Minikube for local Kubernetes development.
- Build custom Docker images and load them into Minikube.
- Author Kubernetes manifest files (YAML) for Deployments and Services.
- Deploy the multi-container application stack on Kubernetes.
- Configure inter-service communication using Kubernetes DNS.
- Verify application health using `kubectl` commands.
- Explore Kubernetes networking concepts including NodePort and ClusterIP.

3 Tools and Technologies Used

- Ubuntu Linux (Host Operating System)
- Docker Engine (Image Build Tool)
- Minikube v1.32+ (Local Kubernetes Cluster)
- `kubectl` (Kubernetes CLI)
- Python 3.11 / Slim (Backend Runtime)
- Nginx Alpine (Frontend Web Server)
- MySQL 8 (Relational Database)
- Kubernetes ConfigMap and Secret objects

4 Background: Key Kubernetes Concepts

Concept	Description
Pod	Smallest deployable unit; wraps one or more containers.
Deployment	Manages a desired number of Pod replicas with rolling-update support.
Service	Stable network endpoint that load-balances traffic to Pods.
ClusterIP	Default Service type; only reachable within the cluster.
NodePort	Exposes a Service on a static port of each cluster node.
ConfigMap	Stores non-sensitive configuration as key-value pairs.
Secret	Stores sensitive data (passwords, tokens) in base64 encoding.
Namespace	Logical partition for isolating resources within a cluster.

5 Implementation Details

5.1 Minikube Installation and Cluster Initialisation

Minikube was installed using the official binary release, and the cluster was started with the Docker driver. The application images were then built locally and loaded into the Minikube runtime:

```
# Install Minikube
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# Install kubectl
sudo snap install kubectl --classic

# Start the local cluster
minikube start --driver=docker

# Build backend image
docker build -t lab2-backend:latest ./backend

# Build frontend image
docker build -t lab2-frontend:latest ./frontend

# Load images into Minikube
minikube image load lab2-backend:latest
minikube image load lab2-frontend:latest

# Verify images are available locally
docker images | grep lab2
```

Listing 1: Minikube installation and cluster start

```
devops/Lab_2/multi-container took 36s
• minikube start
  🐳 minikube v1.38.1 on Ubuntu 24.04
  🌟 Using the docker driver based on existing profile
  🏆 Starting "minikube" primary control-plane node in "minikube" cluster
  🔄 Pulling base image v0.0.50 ...
  🔄 Updating the running docker "minikube" container ...
  🔄 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
  🔄 Verifying Kubernetes components...
    ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
  🌟 Enabled addons: default-storageclass, storage-provisioner
  🏁 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

Figure 1: Minikube cluster successfully started and node in Ready state

```
devops/Lab_2/multi-container took 1m45s
• kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready     control-plane  17d   v1.35.1
```

Figure 2: Kubernetes node status after cluster startup

5.2 Building and Pushing Docker Images

The custom images from Lab 2 were rebuilt locally and loaded into Minikube so that the Kubernetes cluster can use them without a remote container registry.

```
# Point shell to Minikube's Docker daemon
eval $(minikube docker-env)

# Build backend image
docker build -t lab2-backend:latest ./backend

# Build frontend image
docker build -t lab2-frontend:latest ./frontend

# Load images into Minikube
minikube image load lab2-backend:latest
minikube image load lab2-frontend:latest

# Verify images are available locally
docker images | grep lab2
```

Listing 2: Build images inside Minikube's Docker context

```
devops/Lab 2/multi-container
> docker build -t lab2-backend:latest ./backend
[+] Building 2.7s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 530B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 1.9s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:9a7765b36773a37061455b332f18e265e7f58f6fea9c419a5 0.1s
=> => resolve docker.io/library/python:3.11-slim@sha256:9a7765b36773a37061455b332f18e265e7f58f6fea9c419a5 0.1s
=> [internal] load build context                                   0.0s
=> => transferring context: 63B                                       0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt .                             0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                                       0.0s
=> exporting to image                                              0.3s
=> => exporting layers                                              0.0s
=> => exporting manifest sha256:ff7b319772656c34876a4af7b216ce37326bb2764fca94a1622b1dee05d51ad3 0.0s
=> => exporting config sha256:908afd513d61a3713c162f1c9c7a9040065cfb0273ceb573dee1b26def771fc3 0.0s
=> => exporting attestation manifest sha256:4ff04ed75a064fd4da232a04d22b413debb374cf0bdc9b3e9a20d46896cb 0.1s
=> => exporting manifest list sha256:c9bf268af965352e97cf536260a6be8d7cb7c9486cc943247172e55efaa8b012 0.0s
=> => naming to docker.io/library/lab2-backend:latest              0.0s
=> => unpacking to docker.io/library/lab2-backend:latest           0.0s
```

Figure 3: Backend image built inside Minikube's Docker daemon

```
devops/Lab 2/multi-container took 3s
> docker build -t lab2-frontend:latest ./frontend
[+] Building 0.9s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 998B                                0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine     0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 62B                                       0.0s
=> [1/5] FROM docker.io/library/nginx:alpine@sha256:1d13701a5f9f3fb01aaa88cef2344d65b6b5bf6b7d9fa4cf0dca557a8 0.1s
=> => resolve docker.io/library/nginx:alpine@sha256:1d13701a5f9f3fb01aaa88cef2344d65b6b5bf6b7d9fa4cf0dca557a8 0.1s
=> CACHED [2/5] RUN apk add --no-cache openssl && mkdir -p /etc/nginx/certs && openssl req -x509 -nodes -ne 0.0s
=> CACHED [3/5] RUN rm -rf /usr/share/nginx/html/*                0.0s
=> CACHED [4/5] COPY index.html /usr/share/nginx/html/index.html 0.0s
=> CACHED [5/5] COPY nginx.conf /etc/nginx/conf.d/default.conf   0.0s
=> exporting to image                                              0.2s
=> => exporting layers                                              0.0s
=> => exporting manifest sha256:9dd28874bd29caf3fb0fa6a579af56fde09bc9143df40bd200b3f87ceb4084d6 0.0s
=> => exporting config sha256:21c2ed792791f4c3ffa1b2ee63304e2d0cb3a8a83774d84c6277a5cddde1cb4e6 0.0s
=> => exporting attestation manifest sha256:a4fa703ffe51dc362cc4e4e058795d62032f57c5d40e8e2c0b0a50df75ed457b 0.1s
=> => exporting manifest list sha256:3f009e94ff3ac6dce618991f82bcc711f44a4c35ed3b5fee15c047ccb982dff7 0.0s
=> => naming to docker.io/library/lab2-frontend:latest             0.0s
=> => unpacking to docker.io/library/lab2-frontend:latest          0.0s
```

Figure 4: Frontend image built inside Minikube's Docker daemon

5.3 Kubernetes Manifest Files

All Kubernetes resources are defined as YAML manifests organised in a `k8s/` directory and combined with a `kustomization.yaml` file:

```
k8s/
namespace.yaml
configmap.yaml
secret.yaml
database-init-configmap.yaml
database-pvc.yaml
database.yaml
database-service.yaml
backend.yaml
backend-service.yaml
frontend.yaml
frontend-service.yaml
kustomization.yaml
```

5.3.1 MySQL Secret

Sensitive credentials are stored in a Kubernetes Secret rather than plain environment variables:

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
  namespace: lab2
type: Opaque
stringData:
  MYSQL_ROOT_PASSWORD: "rootpassword"
  MYSQL_PASSWORD: "lab2pass"
  DB_USER: "lab2user"
  DB_PASSWORD: "lab2pass"
```

Listing 3: k8s/secret.yaml

5.4 Building and Pushing Docker Images

The custom images from Lab 2 were rebuilt locally and loaded into Minikube so that the Kubernetes cluster can use them without a remote container registry.

```
# Point shell to Minikube's Docker daemon
eval $(minikube docker-env)

# Build backend image
docker build -t lab2-backend:latest ./backend

# Build frontend image
docker build -t lab2-frontend:latest ./frontend

# Load images into Minikube
minikube image load lab2-backend:latest
minikube image load lab2-frontend:latest

# Verify images are available locally
docker images | grep lab2
```

Listing 4: Build images inside Minikube's Docker context

```
devops/Lab 2/multi-container
> docker build -t lab2-backend:latest ./backend
[+] Building 2.7s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 530B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 1.9s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:9a7765b36773a37061455b332f18e265e7f58f6fea9c419a5 0.1s
=> => resolve docker.io/library/python:3.11-slim@sha256:9a7765b36773a37061455b332f18e265e7f58f6fea9c419a5 0.1s
=> [internal] load build context                                   0.0s
=> => transferring context: 63B                                       0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt .                             0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                                       0.0s
=> exporting to image                                              0.3s
=> => exporting layers                                              0.0s
=> => exporting manifest sha256:ff7b319772656c34876a4af7b216ce37326bb2764fca94a1622b1dee05d51ad3 0.0s
=> => exporting config sha256:908afd513d61a3713c162f1c9c7a9040065cfb0273ceb573dee1b26def771fc3 0.0s
=> => exporting attestation manifest sha256:4ff04ed75a064fd4da232a04d22b413debb374cf0bdc9b3e9a20d46896cb 0.1s
=> => exporting manifest list sha256:c9bf268af965352e97cf536260a6be8d7cb7c9486cc943247172e55efaa8b012 0.0s
=> => naming to docker.io/library/lab2-backend:latest              0.0s
=> => unpacking to docker.io/library/lab2-backend:latest           0.0s
```

Figure 5: Backend image built inside Minikube's Docker daemon

```
devops/Lab 2/multi-container took 3s
> docker build -t lab2-frontend:latest ./frontend
[+] Building 0.9s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 998B                                0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine     0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 62B                                       0.0s
=> [1/5] FROM docker.io/library/nginx:alpine@sha256:1d13701a5f9f3fb01aaa88cef2344d65b6b5bf6b7d9fa4cf0dca557a8 0.1s
=> => resolve docker.io/library/nginx:alpine@sha256:1d13701a5f9f3fb01aaa88cef2344d65b6b5bf6b7d9fa4cf0dca557a8 0.1s
=> CACHED [2/5] RUN apk add --no-cache openssl && mkdir -p /etc/nginx/certs && openssl req -x509 -nodes -ne 0.0s
=> CACHED [3/5] RUN rm -rf /usr/share/nginx/html/*                0.0s
=> CACHED [4/5] COPY index.html /usr/share/nginx/html/index.html 0.0s
=> CACHED [5/5] COPY nginx.conf /etc/nginx/conf.d/default.conf    0.0s
=> exporting to image                                              0.2s
=> => exporting layers                                              0.0s
=> => exporting manifest sha256:9dd28874bd29caf3fb0fa6a579af56fde09bc9143df40bd200b3f87ceb4084d6 0.0s
=> => exporting config sha256:21c2ed792791f4c3ffa1b2ee63304e2d0cb3a8a83774d84c6277a5cddde1cb4e6 0.0s
=> => exporting attestation manifest sha256:a4fa703ffe51dc362cc4e4e058795d62032f57c5d40e8e2c0b0a50df75ed457b 0.1s
=> => exporting manifest list sha256:3f009e94ff3ac6dce618991f82bcc711f44a4c35ed3b5fee15c047ccb982dff7 0.0s
=> => naming to docker.io/library/lab2-frontend:latest             0.0s
=> => unpacking to docker.io/library/lab2-frontend:latest          0.0s
```

Figure 6: Frontend image built inside Minikube's Docker daemon

5.5 Kubernetes Manifest Files

All Kubernetes resources are defined as YAML manifests organised in a `k8s/` directory and combined with a `kustomization.yaml` file:

```
k8s/
namespace.yaml
configmap.yaml
secret.yaml
database-init-configmap.yaml
database-pvc.yaml
database.yaml
database-service.yaml
backend.yaml
backend-service.yaml
frontend.yaml
frontend-service.yaml
kustomization.yaml
```


5.5.1 MySQL Secret

Sensitive credentials are stored in a Kubernetes Secret rather than plain environment variables:

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
  namespace: lab2
type: Opaque
stringData:
  MYSQL_ROOT_PASSWORD: "rootpassword"
  MYSQL_PASSWORD: "lab2pass"
  DB_USER: "lab2user"
  DB_PASSWORD: "lab2pass"
```

Listing 5: k8s/secret.yaml

5.5.2 Database StatefulSet and Service

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
  namespace: lab2
spec:
  serviceName: mysql
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          ports:
            - containerPort: 3306
              name: mysql
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: MYSQL_ROOT_PASSWORD
            - name: MYSQL_DATABASE
              valueFrom:
                configMapKeyRef:
```

```

        name: db-config
        key: MYSQL_DATABASE
- name: MYSQL_USER
  valueFrom:
    configMapKeyRef:
      name: db-config
      key: MYSQL_USER
- name: MYSQL_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-credentials
      key: MYSQL_PASSWORD
volumeMounts:
- name: mysql-data
  mountPath: /var/lib/mysql
- name: init-script
  mountPath: /docker-entrypoint-initdb.d
resources:
  requests:
    memory: "256Mi"
    cpu: "250m"
  limits:
    memory: "512Mi"
    cpu: "500m"
livenessProbe:
  exec:
    command:
      - /bin/sh
      - -c
      - mysqladmin ping -h 127.0.0.1 -u root -
        p$MYSQL_ROOT_PASSWORD --protocol=TCP
    initialDelaySeconds: 60
    periodSeconds: 10
    timeoutSeconds: 5
    failureThreshold: 3
readinessProbe:
  exec:
    command:
      - /bin/sh
      - -c
      - mysqladmin ping -h 127.0.0.1 -u root -
        p$MYSQL_ROOT_PASSWORD --protocol=TCP
    initialDelaySeconds: 40
    periodSeconds: 5
    timeoutSeconds: 5
    failureThreshold: 3
volumes:
- name: init-script
  configMap:
    name: db-init-script
volumeClaimTemplates:

```

```
- metadata:
  name: mysql-data
  spec:
    accessModes:
    - ReadWriteOnce
    resources:
      requests:
        storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: mysql
  namespace: lab2
spec:
  clusterIP: None
  ports:
  - port: 3306
    targetPort: 3306
  selector:
    app: mysql
```

Listing 6: k8s/database.yaml and k8s/database-service.yaml

The database runs as a StatefulSet with a PVC, and the headless Service name `mysql` is used for stable internal DNS resolution.

5.5.3 Backend Deployment and Service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: lab2
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
      - name: backend
        image: lab2-backend:latest
        imagePullPolicy: Never # Use local image for development
        ports:
        - containerPort: 5000
        env:
        - name: DB_HOST
```

```

    valueFrom:
      configMapKeyRef:
        name: db-config
        key: DB_HOST
- name: DB_PORT
  valueFrom:
    configMapKeyRef:
      name: db-config
      key: DB_PORT
- name: DB_USER
  valueFrom:
    configMapKeyRef:
      name: db-config
      key: MYSQL_USER
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-credentials
      key: DB_PASSWORD
- name: DB_NAME
  valueFrom:
    configMapKeyRef:
      name: db-config
      key: DB_NAME
resources:
  requests:
    memory: "128Mi"
    cpu: "100m"
  limits:
    memory: "256Mi"
    cpu: "200m"
livenessProbe:
  httpGet:
    path: /api/health
    port: 5000
  initialDelaySeconds: 20
  periodSeconds: 10
  timeoutSeconds: 5
  failureThreshold: 3
readinessProbe:
  httpGet:
    path: /api/health
    port: 5000
  initialDelaySeconds: 10
  periodSeconds: 5
  timeoutSeconds: 5
  failureThreshold: 3

```

```
---
```

```

apiVersion: v1
kind: Service
metadata:

```

```
name: backend
namespace: lab2
spec:
  type: ClusterIP
  ports:
  - port: 5000
    targetPort: 5000
    protocol: TCP
  selector:
    app: backend
```

Listing 7: k8s/backend.yaml and k8s/backend-service.yaml

The backend uses Flask and exposes a health endpoint at `/api/health`.

5.5.4 Frontend Deployment and Service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: lab2
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
      - name: frontend
        image: lab2-frontend:latest
        imagePullPolicy: Never # Use local image for development
        ports:
        - containerPort: 80
        - containerPort: 443
        volumeMounts:
        - name: nginx-config
          mountPath: /etc/nginx/conf.d/default.conf
          subPath: default.conf
      resources:
        requests:
          memory: "64Mi"
          cpu: "50m"
        limits:
          memory: "128Mi"
          cpu: "100m"
      livenessProbe:
        httpGet:
```

```

        path: /
        port: 80
        initialDelaySeconds: 10
        periodSeconds: 10
        timeoutSeconds: 5
        failureThreshold: 3
    readinessProbe:
        httpGet:
            path: /
            port: 80
            initialDelaySeconds: 5
            periodSeconds: 5
            timeoutSeconds: 5
            failureThreshold: 3
    volumes:
    - name: nginx-config
      configMap:
        name: nginx-config
---
apiVersion: v1
kind: Service
metadata:
  name: frontend
  namespace: lab2
spec:
  type: ClusterIP
  ports:
    - name: http
      port: 8080
      targetPort: 80
      protocol: TCP
    - name: https
      port: 443
      targetPort: 443
      protocol: TCP
  selector:
    app: frontend

```

Listing 8: k8s/frontend.yaml and k8s/frontend-service.yaml

The frontend Service is exposed as a `ClusterIP` object and is accessed locally using `kubectl port-forward` during testing.

5.6 Deploying All Resources

All manifests were applied to the cluster using the Kustomize overlay:

```

kubectl apply -k k8s/

# Verify all resources are created
kubectl get all -n lab2
kubectl get pods -n lab2

```

```
kubectl get svc -n lab2
```

Listing 9: Applying all Kubernetes manifests

```
devops/Lab_2/multi-container took 12s
> kubectl apply -k k8s/
namespace/lab2 unchanged
configmap/db-config unchanged
configmap/db-init-script unchanged
secret/db-credentials configured
service/backend unchanged
service/frontend unchanged
service/mysql unchanged
persistentvolumeclaim/mysql-data unchanged
deployment.apps/backend unchanged
deployment.apps/frontend unchanged
statefulset.apps/mysql configured
```

Figure 7: Output of `kubectl apply -k k8s/` showing resources created

```
devops/Lab_2/multi-container took 2s
> kubectl get all -n lab2
```

NAME	READY	STATUS	RESTARTS	AGE
pod/backend-bb77dd447-mkpwl	1/1	Running	5 (7m8s ago)	4d21h
pod/backend-bb77dd447-pkqls	1/1	Running	5 (7m8s ago)	4d21h
pod/frontend-69895478db-8pkp2	1/1	Running	10 (7m18s ago)	17d
pod/frontend-69895478db-gllfn	1/1	Running	10 (6m15s ago)	17d
pod/frontend-dc75b6bd9-wnrsg	0/1	ContainerCreating	0	4d22h
pod/mysql-0	1/1	Running	4 (7m17s ago)	4d21h

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/backend	ClusterIP	10.104.104.245	<none>	5000/TCP	17d
service/frontend	ClusterIP	10.96.24.226	<none>	8080/TCP,443/TCP	17d
service/mysql	ClusterIP	None	<none>	3306/TCP	17d

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/backend	2/2	2	2	4d21h
deployment.apps/frontend	2/2	1	2	17d

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/backend-bb77dd447	2	2	2	4d21h
replicaset.apps/frontend-69895478db	2	2	2	17d
replicaset.apps/frontend-dc75b6bd9	1	1	0	4d22h

NAME	READY	AGE
statefulset.apps/mysql	1/1	4d21h

Figure 8: Output of `kubectl get all -n lab2` after deployment

5.7 Verifying Pod and Service Status

```
# Watch pods come up

# Describe a pod for detailed events
kubectl describe pod <pod-name>

# Check all services
kubectl get services
```

Listing 10: Checking Pod and Service health

```
devops/Lab_2/multi-container
> kubectl get pods -n lab2
```

NAME	READY	STATUS	RESTARTS	AGE
backend-bb77dd447-mkpwl	1/1	Running	5 (7m30s ago)	4d21h
backend-bb77dd447-pkqls	1/1	Running	5 (7m30s ago)	4d21h
frontend-69895478db-8pkp2	1/1	Running	10 (7m40s ago)	17d
frontend-69895478db-gllfn	1/1	Running	10 (6m37s ago)	17d
frontend-dc75b6bd9-wnrsg	0/1	ContainerCreating	0	4d22h
mysql-0	1/1	Running	4 (7m39s ago)	4d21h

Figure 9: All Pods in Running state with STATUS and READY columns

```
devops/Lab_2/multi-container
> kubectl get svc -n lab2
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
backend      ClusterIP   10.104.104.245 <none>        5000/TCP         17d
frontend     ClusterIP   10.96.24.226  <none>        8080/TCP,443/TCP 17d
mysql        ClusterIP   None          <none>        3306/TCP         17d
```

Figure 10: Services listing showing ClusterIP for db/backend and NodePort for frontend

```
kubectl apply -k k8s/
```

```
Verify all resources are created kubectl get all -n lab2 kubectl get pods -n lab2 kubectl
get svc -n lab2
```

6 Findings and Observations

- Kubernetes DNS automatically resolved the `db` hostname within the cluster, enabling the backend Pod to connect to MySQL without any IP address changes.
- Using `imagePullPolicy: Never` was essential to prevent Kubernetes from attempting to pull locally built images from a remote registry, which would result in `ErrImagePull` errors.
- The `depends_on` directive from Docker Compose has no direct Kubernetes equivalent; an *init container* or application-level retry logic is required to handle database readiness.
- Separating sensitive configuration into a `Secret` object is a significant security improvement over plain environment variables in Docker Compose files.
- Scaling the backend Deployment to three replicas demonstrated Kubernetes' load-balancing capability with no application downtime.

7 Technical Challenges Encountered

Issue	Root Cause	Resolution
ErrImagePull on backend Pod	Image not present in the local Minikube Docker context	Set <code>imagePullPolicy: Never</code> and rebuilt inside Minikube Docker context
Backend CrashLoopBackOff	MySQL Pod not yet ready when backend started	Backend retries database connection during startup; verified with <code>kubectl logs</code>
NodePort unreachable from host	Firewall blocking non-standard port	Used <code>minikube service frontend -url</code> to tunnel through Minikube
kubectl version mismatch warning	Snap-installed kubectl version ahead of Minikube API server	Pinned kubectl version to match Minikube's Kubernetes version

8 Comparison: Docker Compose vs. Kubernetes

Feature	Docker Compose	Kubernetes
Target environment	Single host, development	Multi-node, production
Configuration format	<code>docker-compose.yml</code>	Multiple YAML manifests
Service discovery	Service name (bridge network)	CoreDNS within cluster
Scaling	<code>-scale</code> flag	<code>kubectl scale</code> / HPA
Health management	Restart policies	Liveness / Readiness probes
Secret management	Environment variables	Secret objects
Load balancing	Basic round-robin	Service-level load balancing

9 Conclusion

This lab successfully demonstrated the migration of a multi-container application from Docker Compose to Kubernetes. Each service from Lab 2—frontend, backend, and database—was re-deployed as a Kubernetes Deployment paired with an appropriate Service type (ClusterIP or NodePort).

The exercise highlighted core Kubernetes concepts including declarative configuration, built-in DNS-based service discovery, Secrets management, and horizontal scaling. Although Kubernetes introduces greater operational complexity than Docker Compose, it provides the robustness, scalability, and self-healing capabilities required for production cloud-native workloads.